

Reviewer Pack — Minimal Local Indeterminacy in Topological Manifolds vs. Cir...

Entry 6522c59e41ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 03:41:25 UTC

Minimal Local Indeterminacy in Topological Manifolds vs. Circuit Monotone Width

Entry ID: 6522c59e41ed **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 03:41:25 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Indeterminacy Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every n -vertex boolean circuit with monotone width $w(n)$, the minimal local indeterminacy index of the associated topological manifold is bounded by a function $\alpha(w(n))$ such that $\alpha(w(n)) = \Theta(\log n)$ and $\alpha(w(n)/2) = O(\log^{(3/2)} n)$.

Rationale (proposer's reasoning):

Local indeterminacy in algebraic topology measures the complexity of a space at small scales, which could reflect the underlying simplicity or complexity of computational structures. This conjecture proposes that local indeterminacy might be related to circuit monotone width, offering a potential bridge between geometric and computational complexity.

Taxonomy category: manifold-local-indeterminacy-circuit-monotone-width (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c08f486f280df8e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -vertex boolean circuits with monotone width $w(n)$, the minimal local indeterminacy index of the associated topological manifold satisfies $\alpha(w(n)) = \Theta(\log n)$ AND $\alpha(w(n)/2) = O(\log^{(3/2)} n)$. The criterion is falsified if there exists a circuit with monotone width $w(n)$ where $\alpha(w(n)) > \Theta(\log n)$ OR $\alpha(w(n)/2) > O(\log^{(3/2)} n)$, or any seed produces a metric outside the bounds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "local indeterminacy" AND "circuit monotone width" - "topological manifold" AND "boolean circuit complexity" - "indeterminacy index" AND $\Theta(\log n)$ AND boolean circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n <= 1:
            return []
        gate = random.randint(2, n-1)
        inputs = [random.randint(0, 1) for _ in range(gate)]
        circuit = [(gate, inputs)]
        for _ in range(n - gate - 1):
            new_gate = random.randint(2, gate + 1)
            inputs = [random.randint(0, 1) for _ in range(new_gate)]
            circuit.append((new_gate, inputs))
            gate += new_gate
        return circuit

    def calculate_local_indeterminacy(circuit):
        if not circuit:
            return 0
        n = len(circuit)
        local_indeterminacy = [0] * n
        for i in range(n):
            gate, inputs = circuit[i]
            for j in range(gate):
                if inputs[j] == 1:
                    local_indeterminacy[i] += 1
```

```

        return max(local_indeterminacy)

def monotone_width(circuit):
    width = 0
    for gate, _ in circuit:
        width = max(width, gate)
    return width

n_max = 40
instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    circuit = generate_circuit(n)
    w_n = monotone_width(circuit)
    local_indeterminacy = calculate_local_indeterminacy(circuit)
    alpha_w_n = math.log(n) if n > 0 else 0
    alpha_w_n_over_2 = math.log((n // 2)) if (n // 2) > 0 else 0

    metric_values.append(local_indeterminacy)

mean_value = sum(metric_values) / len(metric_values)
std_value = math.sqrt(sum((x - mean_value) ** 2 for x in metric_values) / len(metric_values))

conjecture_holds = all(alpha_w_n <= math.log(n) and alpha_w_n_over_2 <= math.log((n // 2)) for
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "local_indeterminacy",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):

```


#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	8993

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 167912 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/6522c59e41ed.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6522c59e41ed.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6522c59e41ed.tar.gz (if generated)